

Actua como asistente de programacion para una practica de Estructura de Datos en Python.

Contexto:

Ya tengo un proyecto desarrollado en clase con Python y programacion orientada a objetos. En el proyecto ya existe una clase Stack o Pila y dos aplicaciones que utilizan esta estructura. Ahora necesito agregar una nueva estructura de datos: Cola o Queue.

Objetivo:

Necesito crear una clase Queue con las operaciones basicas enqueue, dequeue, printQueue, firstQueue

y lastQueue. La clase debe integrarse al mismo proyecto existente sin modificar lo que ya funciona.

Instrucciones importantes:

1. No generes un proyecto nuevo desde cero.
2. No reemplaces ni modifiques la clase Stack existente.
3. No elimines las aplicaciones ya desarrolladas con pila.
4. Respeta la organizacion actual de archivos, clases y nombres del proyecto.
5. La solucion debe estar hecha con programacion orientada a objetos.
6. Propón cambios minimos e indica exactamente en que archivo y en que parte deben colocarse.
7. La clase Queue debe tener al menos los metodos enqueue, dequeue, printQueue, firstQueue y lastQueue.
8. Si se requiere un metodo auxiliar para saber si la cola esta vacia, proponlo sin afectar la lista de operaciones solicitadas.
9. Explica brevemente como funciona FIFO y como se diferencia de una pila.
10. Proporciona pruebas para comprobar que la clase funciona correctamente.

Codigo existente:

```
[
from estructuras.lineales.nodo import Node
class Stack(object):
    def __init__(self):
        self.top = None

# pregunta si la pila esta vacia o no
def is_empty(self):
```

```

        return self.top is None

def push(self, data):

    nuevo = Node(data)
    nuevo.next = self.top

    self.top = nuevo

def pop(self):
    if self.top is None:
        return None

    valor = self.top.data
    self.top = self.top.next

    # Paso 4: Retornar el valor extraído (como dice la flecha verde:
    'return valor')
    return valor

def top_of_stack(self):
    # Verificar si la pila no tiene elementos
    if self.top is None:
        return None

    # Retorna el valor del nodo en la cima pero SIN quitarlo de la pila
    return self.top.data

def print_stack(self):
    # Verificar si está vacía
    if self.top is None:
        return None

    temp = self.top
    print("--- Cima de la Pila ---")
    while temp is not None:
        print(f"[ {temp.data} ]")
        temp = temp.next
    print("-----")

```

```
[class Queue(object):
def __init__(self):
self.first = None
self.last = None
def enqueue(self, valor):
# Agrega el valor al final de la cola
pass
def dequeue(self):
# Elimina el primer elemento que entro
if self.isEmpty():
return None
pass
def firstQueue(self):
# Consulta el primer elemento sin eliminarlo
if self.isEmpty():
return None
pass
def lastQueue(self):
# Consulta el ultimo elemento sin eliminarlo
if self.isEmpty():
return None
pass
def printQueue(self):
# Devuelve el contenido completo de la cola
pass
def isEmpty(self):
return self.fist == None]
```

```
[
import sys
from PyQt5.QtWidgets import QApplication
from load.load_MenuPrincipal import MenuPrincipal

def main():
    app = QApplication(sys.argv)

    ventana = MenuPrincipal()
    ventana.show()
    sys.exit(app.exec_())

if __name__ == '__main__':
    main()
]
```

```
[
from PyQt5.QtWidgets import QMainWindow
from PyQt5 import uic
from load.load_interfaz import LoadInterfaz
from load.load_stack import DialogoPila
from load.load_convercion import LoadInterfazPila
from load.load_queue import LoadInterfazQueue

class MenuPrincipal(QMainWindow):
    def __init__(self):
        super().__init__()
        self.ui = uic.loadUi('ui/MenuPrincipal.ui', self)

        # Conectar acción del menú
        self.actionLista_Enlazada.triggered.connect(self.abrir_interfaz)
        self.actionPila_2.triggered.connect(self.abrir_stack)
        self.actionconvercion_infija_posfija.triggered.connect(self.abrir_convercion)
        self.actionQueue.triggered.connect(self.abrir_queue)

    def abrir_interfaz(self):
        self.interfaz = LoadInterfaz()
        self.interfaz.exec()

    def abrir_stack(self):
        self.stack = DialogoPila()
]
```

```

        self.stack.exec()

    def abrir_convercion(self):
        self.convercion = LoadInterfazPila()
        self.convercion.exec()

    def abrir_queue(self):
        self.queue = LoadInterfazQueue()
        self.queue.exec()

```

]

Resultado esperado de tu respuesta:

- Primero analiza la estructura del código existente.
- Después propone la clase Queue.
- Luego indica cómo integrarla al proyecto sin afectar Stack ni las aplicaciones existentes.
- Finalmente proporciona una prueba guiada de enqueue, dequeue, printQueue, firstQueue y lastQueue.
- No omitas explicación, porque debo entender el código para defenderlo en clase.

¡Hola! Qué gran proyecto tienes entre manos. Integrar una nueva estructura de datos en un sistema que ya utiliza interfaces gráficas (como PyQt5) es un excelente ejercicio. Lo mejor de todo es que tu arquitectura actual basada en componentes (`estructuras.lineales`) facilita mucho las cosas, ya que está diseñada para ser modular.

Vamos a analizar tu código paso a paso, entender los conceptos clave y construir la solución exacta sin romper nada de lo que ya tienes programado.

1. Análisis de la estructura del código existente

Al revisar los fragmentos de código que compartiste, podemos notar lo siguiente:

- **Arquitectura modular:** Tienes las estructuras separadas de la lógica de la interfaz. Tu clase `Stack` importa un nodo dinámico mediante `from estructuras.lineales.nodo import Node`. Esto significa que tu `Queue` (Cola) debe seguir la misma regla y reutilizar ese mismo nodo.
- **Preparación en la UI:** Tu archivo `MenuPrincipal.py` ya **tiene todo listo** para recibir la cola. Nota que ya importa `from load.load_queue import LoadInterfazQueue` y conecta el botón con `self.actionQueue.triggered.connect(self.abrir_queue)`.

- **El esqueleto de la Cola:** El fragmento de la clase `Queue` que te dieron tiene un par de detalles a corregir (un error de dedo en `self.fist == None` en lugar de `self.first` y falta de importación del `Nodo`), los cuales resolveremos a continuación.

2. Explicación conceptual: FIFO vs. LIFO (Cola vs. Pila)

Para defender tu examen o práctica en clase, debes tener clarísima esta diferencia:

- **Pila (Stack) - El principio LIFO (*Last In, First Out*):** El último elemento en llegar es el primero en salir. Imagina una **pila de platos**: el último que lavas y pones arriba es el primero que retiras para usar. En tu código, el puntero `top` siempre mira al último elemento agregado.
- **Cola (Queue) - El principio FIFO (*First In, First Out*):** El primer elemento en llegar es el primero en salir. Imagina la **fila del banco**: la primera persona que llega es la primera en ser atendida. Las inserciones se hacen por el final (`last`) y las extracciones se hacen por el principio (`first`).

3. Propuesta de la clase `Queue`

python

```
from estructuras.lineales.lista_enlazada_simple import LinkedList
```

```
class Queue:
```

```
    def __init__(self):
```

```
        self.lista = LinkedList()
```

```
        self.resultado = ""
```

```
    def enqueue(self, data):
```

```
        # Insertar al final → comportamiento FIFO
```

```
        self.lista.insert_at_end(data)
```

```
        self.resultado = f"Elemento {data} agregado a la cola."
```

```
        return self.resultado
```

```
    def dequeue(self):
```

```
        # Eliminar al inicio → comportamiento FIFO
```

```
if self.lista.head is None:

    self.resultado = "La cola está vacía, no se puede eliminar."

    return None

eliminado = self.lista.head.data

self.lista.delete_first()

self.resultado = f"Elemento {eliminado} eliminado de la cola."

return eliminado
```

```
def firstQueue(self):

    # Primer elemento sin eliminar

    if self.lista.head is None:

        self.resultado = "La cola está vacía."

        return None

    self.resultado = f"El primer elemento es: {self.lista.head.data}"

    return self.lista.head.data
```

```
def lastQueue(self):

    # Último elemento sin eliminar

    if self.lista.head is None:

        self.resultado = "La cola está vacía."

        return None

    temp = self.lista.head

    while temp.next is not None:

        temp = temp.next

    self.resultado = f"El último elemento es: {temp.data}"

    return temp.data
```

```
def printQueue(self):

    # Mostrar todos los elementos
```


- **Cola (Queue):** sigue el principio **FIFO** (*First In, First Out*). El primer elemento en entrar es el primero en salir. Ejemplo: una fila en el banco.

5. Pruebas guiadas

En un archivo de pruebas (puede ser `test\_queue.py` en la raíz del proyecto):

```
``python
from estructuras.lineales.queue import Queue

cola = Queue()

print("Prueba enqueue:")
cola.enqueue(10)
cola.enqueue(20)
cola.enqueue(30)
print(cola.printQueue()) # 10 -> 20 -> 30

print("\nPrueba firstQueue:")
print(cola.firstQueue()) # 10

print("\nPrueba lastQueue:")
print(cola.lastQueue()) # 30

print("\nPrueba dequeue:")
print(cola.dequeue()) # 10
print(cola.printQueue()) # 20 -> 30
```

```
print("\nPrueba vaciar cola:")
cola.dequeue()
cola.dequeue()
print(cola.printQueue()) # La cola está vacía.
'''

---
```

☒ Con esto tu clase `Queue` queda lista, integrada al proyecto, y puedes defenderla en clase explicando que:

- Usaste ****POO****.
- Respetaste la organización del proyecto.
- No tocaste la pila ni sus aplicaciones.
- Implementaste correctamente el comportamiento ****FIFO****.

¿Quieres que te prepare también la clase `LoadInterfazCola` en PyQt5 para que tengas la GUI lista igual que con la pila?